Xu

Xu Noodles
The Complete Guide

How the ordering website and the owner's Xu Kitchen app work, how to set them up from scratch, and how to keep them running smoothly.

Who this guide is for

🍜 The owner
Understand what the app does and what to do when something isn't working.

🛠️ A helper setting it up
Step-by-step setup with no coding, from new accounts to the first order.

💻 A future developer
How the parts fit together, the database files, and the safety rules.

Based on the project documentation in the repository (README.md and the docs folder).
This guide contains no passwords, keys or customer information. Keep it that way: never write them into this document or the code.

Contents

1 What the app does — Customers and the owner, at a glance

2 How it all fits together — A simple picture of the system

3 Setting up from scratch — Step by step, no coding needed

4 The database files — What to run, in which order, and what never to re-run

5 Who can see and change what — Security in plain words

6 Making changes and going live — Testing, publishing and undoing a release

7 When something goes wrong — Fixes for common problems

8 Words explained — A short glossary

The whole system in one paragraph

Customers open the Xu Noodles website on their phone, choose dishes and place a delivery order that they pay for on delivery (cash or UPI). The order goes into an online database. The owner's phone shows it instantly with a chime and a notification. The owner accepts it with a cooking time (or rejects it with a reason), and the customer's screen updates within seconds. The website is hosted on **Vercel**; the database, owner login and notifications run on **Supabase**.

1 What the app does

In short

There are two parts: a **website for customers** to order food, and an **owner screen** (the "Xu Kitchen" app) to run the kitchen. The restaurant is delivery only, within about 3 km of Gaur City 2 (Greater Noida West). There is no online payment and customers don't need an account.

📱

For customers

- **Menu** with categories, search, a "Veg only" filter, Veg / Non-veg labels, spice level and Half / Full portions.
- **Offers** show the old price crossed out and a badge like "20% off".
- **Cart** remembers dishes on the phone and shows every charge clearly.
- **Checkout** asks for name, mobile number and address (society, tower, flat, landmark) and remembers them for next time.
- **Order status** page updates by itself every few seconds.

👩‍🍳

For the owner (Xu Kitchen)

- **Orders:** new orders appear instantly with a chime. Choose 30, 40 or 50 minutes, then **Accept**; or **Reject** with a reason. Print a bill. Cancel if delivery fails.
- **Menu:** mark a dish sold out, add, edit or remove dishes.
- **Offers:** create discounts with an end date; stop them any time.
- **Sales:** today, yesterday, this week or month, compared with before.
- **Alerts:** a phone notification for every new order, even when the app is closed.

What a customer pays

Line on the bill	What it means
Subtotal	Menu prices of everything in the cart.
Offer discount	Any offers that apply (shown only when there is one).
GST (5%)	5% of the subtotal after the discount, rounded to the rupee.
Delivery fee	A fixed amount set by the owner (shown only if it isn't ₹0).
Total	**Paid on delivery, by cash or UPI.**

The life of an order

1. **Placed.** The customer taps "Place order · Pay ₹X on delivery". The order is saved as *waiting*.
2. **The owner is told.** The Orders screen chimes every 10 seconds, and the owner's phone gets a notification. After 3 minutes the card flashes and the chime gets louder.

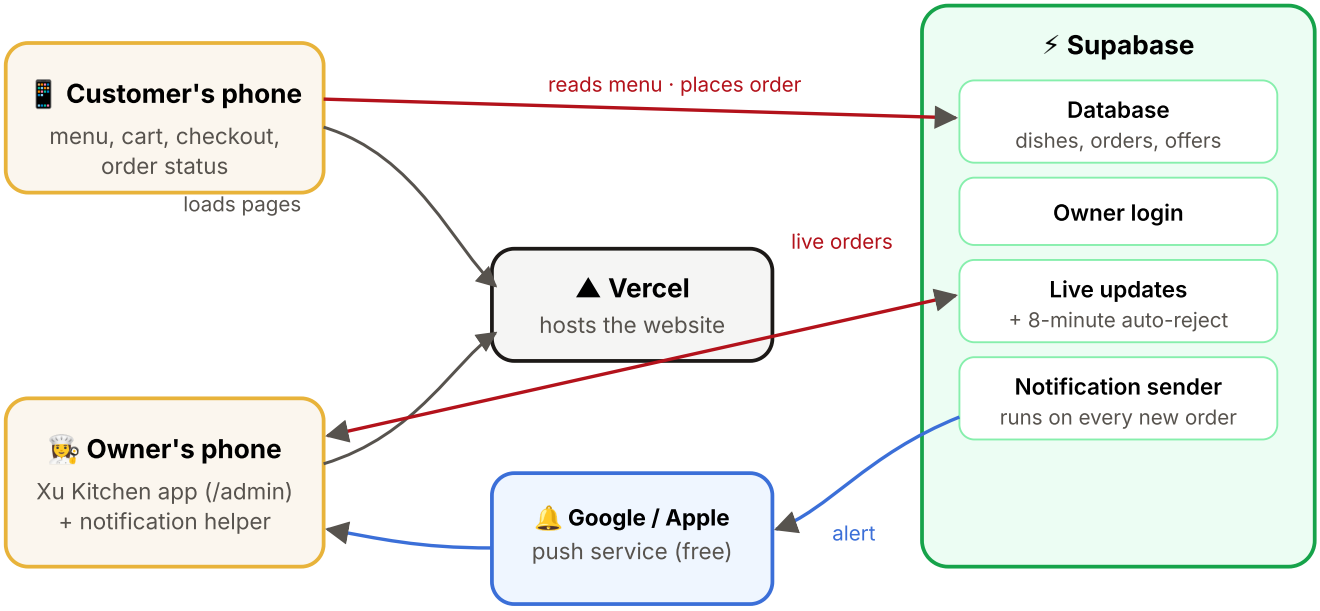
3. **Accepted or rejected.** The owner picks a cooking time and taps Accept, or rejects with a reason (for example "Item sold out", which also hides that dish until 6:00 am).
4. **No answer?** If nobody responds within **8 minutes**, the order is rejected automatically with "Restaurant didn't respond", so the customer isn't left waiting.
5. **Delivered.** The customer sees "Arriving in about X minutes" counting down. If delivery fails (customer not home, refused to pay), the owner marks the order **cancelled**.

> **Good to know**
> All times and "today / this week" boundaries use Indian time (IST), whatever the phone's own settings are.

2 How it all fits together

In short

Phones open the website from **Vercel**. Everything that needs to be stored (dishes, orders, offers, the owner's login) lives in **Supabase**. When a new order arrives, Supabase sends a notification to the owner's phone through Google's or Apple's free push service.



Who does what

Part	Its job
Vercel	Builds the website from the code on GitHub and serves it. The `main` version is the live site; the `dev` version gets its own test link.
Customer's phone	Shows the menu and works out prices for display. The final price is always recalculated by the database, so it can't be faked.
Owner's phone	Runs the Xu Kitchen app: live orders, chime, keeps the screen awake on the Orders tab.
Notification helper (a "service worker")	A small background helper on the owner's phone. It shows order notifications and opens the app when one is tapped. It never stores copies of pages, so updates always show straight away.
Supabase database	Stores everything and enforces the rules (who can see what, prices, checks). Also runs a job every minute that rejects orders nobody answered within 8 minutes.

Notification sender (Edge Function "notify-new-order")	Triggered by every new order. Sends "🔔 New order #N · Collect ₹X" to each registered owner phone.

> **Alerts stay on by themselves**
>
> Every time the owner opens the app, it quietly checks that this phone is still registered for notifications and fixes it if not, with no button to press. The header then shows 🔔 **Alerts on**. If the phone renews its registration in the background, the helper saves the new one automatically.

3 Setting up from scratch

In short

About an hour, no coding. You create a Supabase project, run the database files, create the owner's login, put the website on Vercel, and set up notifications. Keep a password manager open: you'll create a few passwords and keys.

Golden rule

Never paste keys or passwords into GitHub, chat messages, emails or documents (including this one). The code repository is public.

You will need

- [ ] A GitHub account with the project's code
- [ ] A Supabase account (free plan is fine to start)
- [ ] A Vercel account, signed in with GitHub (free plan is fine to start)
- [ ] The owner's phone: Android with Chrome, or iPhone with iOS 16.4 or newer

Step 1: Create the Supabase project

1. In Supabase, click **New project** and name it (for example *xu-noodles*).
2. Let Supabase generate the **database password** and save it in your password manager. It can't be recovered later.
3. Choose the region **South Asia (Mumbai)**, the closest to Greater Noida, so the app is fast.
4. Click **Create new project** and wait a minute or two.

Step 2: Build the database

Open **SQL Editor** in the left menu. For each file below: **New query** → paste the file's contents (in GitHub, open the file → *Copy raw file*) → **Run** → check it says "Success". Do them **once each, in this order**:

1. **Starting tables:** the SQL block in `docs/database.md` (only for a brand-new project).
2. `seed.sql`: the 32 dishes of the menu.
3. `updates.sql`: delivery-only setup. **Deletes all orders** Fine on a new project, but never run it again later.
4. `02_free_text_society.sql`
5. `03_owner_orders.sql`. If it says *pg_cron* isn't available: Database → Extensions → turn on **pg_cron**, then run it again.
6. `04_menu.sql`
7. `05_push.sql`

8. `06_offers_and_sales.sql`

Then set the money settings: **Table Editor → restaurant_settings** → edit the single row: `delivery_fee` and `min_order` in rupees (0 means none).

Step 3: The owner's login

1. **Authentication → Sign In / Providers:** turn **OFF** "Allow new users to sign up". Keep the Email provider on.
2. **Authentication → Users → Add user → Create new user:** enter the owner's email and a strong password, tick **Auto Confirm User**, save.
3. Give the owner the password privately. They can change it later in the app under **Settings → Change password**.

> **Why sign-ups must be off**
>
> Anyone who can sign in is treated as staff and can see customers' addresses and phone numbers. With sign-ups off, only the login you create can get in.

Step 4: Notification keys

Order alerts need a matching pair of keys: a **public** key and a **private** key. Create them once and store both in your password manager. Ask a developer to run this (or run it yourself if Node.js is installed):

```
npx web-push generate-vapid-keys
```

It prints a **Public Key** (about 87 characters, starting with "B") and a **Private Key** (about 43 characters). The private key goes **only** into Supabase (Step 6).

Step 5: Put the website on Vercel

1. In Vercel: **Add New... → Project** → import the GitHub repository. It detects Next.js by itself.
2. Before deploying, open **Environment Variables** and add the three below. Tick **Production**, **Preview** and **Development** for each.
3. Click **Deploy**. When it's done, open the site: the menu should appear.
4. If you change a variable later: **Deployments → ··· → Redeploy** (changes only apply after a new build).

Variable name (exactly)	Where to find the value
`NEXT_PUBLIC_SUPABASE_URL`	Supabase → Project Settings → API → *Project URL*
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	Supabase → Project Settings → API Keys → *publishable* key (called "anon public" on older projects)

`NEXT_PUBLIC_VAPID_PUBLIC_KEY`	The **public** key from Step 4

> **Never add to Vercel**
> The notification **private** key, or Supabase's **service role / secret** key. The three values above are designed to be public; those two are not.

Step 6: Supabase secrets for notifications

Supabase → **Edge Functions → Secrets**. Add these three:

Secret name	Value
`VAPID_PUBLIC_KEY`	The public key from Step 4 (the same as in Vercel)
`VAPID_PRIVATE_KEY`	The private key from Step 4
`VAPID_SUBJECT`	The word `mailto:` followed directly by the restaurant's contact email

Supabase provides the project address and its own service key to the function automatically; don't add them.

Step 7: The notification sender

1. Supabase → **Edge Functions → Deploy a new function → Via Editor**.
2. Name it exactly `notify-new-order`.
3. Delete the example code, paste the whole file `supabase/functions/notify-new-order/index.ts` from GitHub, and click **Deploy**.
4. **Verify JWT** (in the function's settings): it can stay on. If your publishable key starts with `sb_publishable_`, turn it **off**, so phones can re-register while the app is closed. This is safe: the function checks every request itself.

Whenever that file changes in GitHub, paste the new version here and deploy again.

Step 8: Send alerts on every new order (Database Webhook)

1. Supabase → **Database → Webhooks** (newer dashboards: Integrations → Database Webhooks). Click **Enable webhooks** if asked.
2. **Create a new hook:** name `new_order_push` · table `orders` · event **Insert** only · type **Supabase Edge Functions** · function `notify-new-order` · method **POST**.
3. Under HTTP Headers, click **Add auth header with service key**, then **Create webhook**.

Step 9: Set up the owner's phone

🤖 Android (Chrome)

1. Open *your-site*/admin and sign in. Orders opens.
2. Chrome menu ⋮ → **Install app**; open **Xu Kitchen** from the home screen.
3. On the blue banner, tap **Turn on alerts** → **Allow**. The header shows 🔔 **Alerts on**.
4. **Settings → Send test notification.**
5. Allow Chrome / Xu Kitchen to run in the background (battery settings).

🍎 iPhone (iOS 16.4+)

1. Open *your-site*/admin in **Safari**.
2. **Share → Add to Home Screen → Add.**
3. Open **Xu Kitchen** from the home screen and sign in (it has its own sign-in).
4. Tap **Turn on alerts** → **Allow**.
5. **Settings → Send test notification.**

> **Sound**
>
> Phones only allow sound after the screen has been touched. If a yellow banner says **"Tap anywhere to turn on order sound"**, tap once anywhere.

Step 10: Final check

- [ ] The menu shows on the home page.
- [ ] A test order from another phone appears on the Orders screen with a chime.
- [ ] The owner's phone gets "🔔 New order #N · Collect ₹X", even with the app closed.
- [ ] Accepting the order updates the customer's status page within a few seconds.
- [ ] An order left alone is rejected automatically after 8 minutes.
- [ ] Sign-ups are off in Supabase Authentication.

Test orders can't be deleted (by design). Reject or cancel them so they don't count as sales.

4 The database files

In short

The database is built by eight files in the `supabase` folder, run once each **in order**. One of them deletes all orders. Several must never be run again later, because they would put an older version of the ordering logic back.

#	File	What it does	Deletes data?	Run again later?
0	Starting tables (in docs/database.md)	Creates the three original tables. Only for a brand-new project.	No	Only if empty
1	`seed.sql`	Adds the 32 original dishes (skips ones that already exist).	No	Not once the owner edits the menu
2	`updates.sql`	Delivery-only setup: settings table, address fields, order numbers, statuses.	YES, all orders	Never
3	`02_free_text_society.sql`	Customers type their society name.	No	No
4	`03_owner_orders.sql`	Live updates, owner permissions, 8-minute auto-reject.	No	Safe
5	`04_menu.sql`	"Remove from menu", owner can add dishes.	No	No
6	`05_push.sql`	The list of phones that get order alerts.	No	Safe
7	`06_offers_and_sales.sql`	Offers at checkout and the Sales report.	No	Safe

Why some files must not be re-run

Files 2, 3, 5 and 7 each install a new version of the database's **"place order"** logic, each building on the one before. Running an older file after a newer one quietly puts the old version back. For example, re-running `04_menu.sql` after `06_offers_and_sales.sql` removes offers and **stops checkout from working**.

Fix if it happens: run `06_offers_and_sales.sql` again; it restores the current version.

> **Making database changes in future**
>
> Add a new numbered file (`07_…sql`) rather than editing one that has already been run. At the top, write what it does, whether it deletes data and whether it's safe to re-run.

What the database holds

Table	Holds
`menu_items`	Dishes, prices (single or Half / Full), veg / non-veg, sold-out time, "removed from menu" flag
`offers`	Discounts with start and end times
`restaurant_settings`	One row: delivery fee, minimum order, default prep time
`orders`	Each order: number, customer, address, totals, status, reason
`order_items`	The dishes in each order, with the price at the time of ordering
`push_subscriptions`	The owner phones that receive order alerts

Settings the owner can change

In Supabase → **Table Editor → restaurant_settings** (the single row):

- `delivery_fee` : added to every order (₹). 0 = free delivery.
- `min_order` : minimum order value before discounts (₹). 0 = no minimum.

The GST rate (5%) is fixed in two places in the code that must always match: the database's place-order logic and the website's price rules.

5 Who can see and change what

In short

Customers can read the menu, offers and delivery charges, place an order and check its status, and nothing else. They can never see other people's orders, addresses or phone numbers. The signed-in owner can manage orders, the menu, offers and settings. Prices are always checked by the database, so nobody can change what they pay.

Information	Customer (not signed in)	Owner (signed in)
Menu	**Read**	**Read, add, change, sold out, remove** (dishes are never deleted)
Offers	**Read**	**Read, create, stop**
Delivery fee & minimum	**Read**	**Read, change**
Orders	**No access** Can only place an order, and see the status of their own order, without the address or phone	**Read, accept, reject, cancel**
Sales report	**No**	**Yes**
Alert phones list	**No**	**Read, add, remove**

Where secrets are kept

Secret	Kept in	Never in
Notification private key	Supabase secrets	GitHub, Vercel, the website
Supabase service key	Supabase (automatic)	Anywhere else
Owner's password	Supabase login system	Anywhere else
Database password	Your password manager	Anywhere else

The project address, the *publishable* key and the notification *public* key are meant to be public. They are stored in Vercel's settings, not in the code.

Three safety rules that matter most

1. **Sign-ups stay off.** Every signed-in person is treated as staff.
2. **Nothing secret in GitHub.** The repository is public: no keys, passwords, emails or customer data.
3. **Prices come from the database.** The website's prices are only for display; the database recalculates the real total.

What's saved on phones

To be convenient, the app remembers a few things on each phone: the cart, the customer's name and address, and recent orders. On the owner's phone it also keeps the login and the alert registration. Every saved item is checked when it's read. Anything old, broken or pointing to an order that no longer exists is simply thrown away, so it can never crash the app.

6 Making changes and going live

In short

Changes go to the **dev** version first and are tested on its own link. When everything works, a **pull request** copies them to **main**, which is the live site. If a release goes wrong, Vercel can switch back to the previous version in seconds.

Version	Where it appears	Who uses it
`dev`	A preview link from Vercel	You, for testing
`main`	The real website	Customers and the owner

The test link uses the real database

Orders placed on the dev link are **real orders**: they appear on the owner's screen and in Sales. Reject or cancel test orders afterwards. Also, database files run in Supabase affect the live site **immediately**.

Finding the dev link

Vercel → your project → **Deployments** → filter by branch **dev** → open the newest → **Visit**. If Vercel asks you to log in, that's its protection for test links.

Testing checklist (after every change)

Customer side

- [ ] Menu, categories, Veg only and search work
- [ ] Offers show crossed-out prices (if any are active)
- [ ] Cart shows subtotal, discount, GST, fee, total
- [ ] Place order → "Waiting for the restaurant to confirm..."

Owner side

- [ ] Opens straight on Orders; the test order chimes
- [ ] Choose a time → Accept → customer sees "Arriving in about X minutes"
- [ ] Print bill works; Menu, Offers, Sales load
- [ ] Header shows 🔔 Alerts on; the test notification arrives

Going live

1. **Database first.** If the change includes a new database file, run it in Supabase before going live, and make sure the current live site still works with it.
2. On GitHub: **Pull requests → New pull request** → base `main`, compare `dev` → **Create pull request**. Describe what changes for customers and for the owner.

3. Check that Vercel's check is green, then click **Merge pull request**. Vercel updates the live site automatically within a few minutes.
4. Open the live site and place one test order (then cancel it).

Undoing a bad release (rollback)

1. Vercel → your project → **Deployments**.
2. Find the last **Production** version that worked.
3. ⋯ → **Instant Rollback** (or *Promote to Production*). The live site switches back within seconds.
4. Fix the problem on dev, test it, and go live again with a new pull request.

Two things a rollback does NOT undo

- **Database changes.** Anything run in Supabase stays. Don't roll the website back to a version from *before the offers update* while the database has it: every order would fail with "Prices changed...". Roll back only to a version that includes offers.
- **The notification sender.** To undo it, paste its previous code (from GitHub's history) and deploy again.

7 When something goes wrong

> **In short**
> Most problems are one of three things: notifications not arriving, an error message after an update, or old data on one particular phone. Work through the matching checklist below.

🔕 Order alerts not arriving

1. **Look at the owner screen's header and banners.**
   - 🔔 **Alerts on:** the phone is registered; go to step 3.
   - **Blue "Turn on alerts for new orders":** tap **Turn on alerts** → **Allow**.
   - **Red "Order alerts are off":** follow the instruction it shows, then tap **I've allowed it**.
     Android: tap the icon left of the web address → Site settings → Notifications → Allow (or phone Settings → Apps → Chrome / Xu Kitchen → Notifications).
     iPhone: Settings → Notifications → Xu Kitchen → Allow. Alerts only work in the home-screen app, not in Safari.
2. **Settings → Send test notification.** It arrives? Then go to step 4. "Not set up correctly"? Then go to step 5.
3. **Phone settings:** battery saver can delay alerts (allow background use); Do Not Disturb / Focus silences them.
4. **The trigger:** Supabase → Database → Webhooks: `new_order_push` exists, on `orders`, event Insert. Edge Functions → notify-new-order → Logs should show a call for each new order.
5. **The sender:** its three secrets exist, and `VAPID_PUBLIC_KEY` is **exactly** the same as the public key in Vercel. After changing keys, redeploy the site; phones re-register by themselves when the app is next opened.
6. **The list of phones:** Table Editor → `push_subscriptions` should have a row for each phone with alerts on.

⚠️ "Something went wrong. Please refresh." after an update

1. **Refresh once.** The phone may still be showing the previous version.
2. **Only one section shows the message?** The rest of the page keeps working; note which part it is.
3. **Does it work in a private / incognito window?** If yes, it's old data on that phone (see below).
4. **Did the update build properly?** Vercel → Deployments → newest → Build Logs.
5. **Details for the developer:** on a computer, right-click → Inspect → Console. A line starting with `Error in …` names the broken part.
6. **Broken for everyone?** Roll back (Chapter 6), then fix on dev.

📦 Old data saved on one phone

The app already throws away saved data it can't use, so this is rare. If one phone still misbehaves while a private window works, clear that site's data:

Phone	How
Android Chrome	Open the site → icon left of the address → **Site settings → Clear & reset**
iPhone Safari	**Settings → Safari → Advanced → Website Data** → find the site → swipe to delete
iPhone home-screen app	Delete the app from the home screen and add it again

This signs the owner out, empties the cart and forgets the saved address. Alerts re-register by themselves after signing in again.

Other common problems

What you see	Why	What to do
Customer sees "Prices changed because an offer just ended..."	An offer started or ended while they were checking out	Nothing: the cart is kept with the new prices; they just order again
Every order fails with "Prices changed..."	The website is older than the database, or an old database file was re-run	Make sure the live site includes offers; re-run `06_offers_and_sales.sql`
"Sorry, we couldn't load the menu"	Vercel settings missing or wrong, or the Supabase project is paused	Check the three Vercel variables and redeploy; restore a paused project in Supabase
Orders never auto-rejected	*pg_cron* not turned on	Database → Extensions → enable pg_cron, re-run `03_owner_orders.sql` (safe)
New orders take up to 30 s to appear	The live connection dropped; a backup refresh takes over	Reopen the app; check `03_owner_orders.sql` was run
Owner can't sign in	Wrong password or unconfirmed user	Supabase → Authentication → Users: reset the password
Sales tab: "Can't load sales"	`06_offers_and_sales.sql` not run	Run it (safe to re-run)
No sound on the owner's phone	Sound is locked until the screen is touched, or the phone is silent	Tap once anywhere; check media volume
Screen turns off on the Orders tab	The browser can't keep the screen awake	Set a longer screen timeout on the phone

8 Words explained

Word	Meaning
Supabase	The online service that stores the data, handles the owner's login and sends live updates.
Vercel	The service that puts the website online.
GitHub / repository	Where the code is kept. This one is public, so it must never contain secrets.
Branch: dev / main	Two versions of the code. *dev* is for testing; *main* is the live site.
Pull request	A request on GitHub to copy the tested changes from dev into main (going live).
Deployment / rollback	A published version of the website / switching back to an earlier one.
Environment variable	A named setting stored in Vercel (not in the code), such as the database address.
Secret	A private setting stored in Supabase, such as the notification private key.
SQL file	A set of instructions that builds or changes the database, run in Supabase's SQL Editor.
Edge Function	A small program that runs on Supabase. Here it sends the order notifications.
Database Webhook	A rule that runs the Edge Function automatically whenever a new order is saved.
Push notification / VAPID keys	A phone alert sent over the internet / the key pair that proves the alert comes from this app.
Service worker	A small helper on the owner's phone that shows notifications even when the app is closed.
Row level security	Database rules that decide who can read or change each kind of information.
pg_cron	A timer inside the database; it rejects unanswered orders every minute.
Publishable key	A public key that lets the website read the menu and place orders. Safe to be visible.

Where to find more detail

This guide summarises the documentation in the code repository: `README.md`, `docs/setup.md`, `docs/database.md`, `docs/architecture.md` and `docs/operations.md`. A developer should read those for the exact technical details.